MASTER PLAN: Contract-as-Neuron Abstraction + Value Pointers + Fulfilment + ARC-AGI-3 Validation

MASTER PLAN: Contract-as-Neuron Abstraction + Value Pointers + Fulfilment + ARC-AGI-3 Validation

Date: 2026-07 (current session) **Branch context**: arc-lean **Status**: Authoritative execution plan. All prior discussion (cache per-id limits, pointer value surfacing, self-fulfilling contract returns until "equal", every primitive as contract neuron, debloat targets, strong inheritable Contract base type) is captured here and will be executed to completion.

Success Target: End-to-end on ARC-AGI-3 style tasks (small abstraction/reasoning grid programs) using the substrate as a uniform neuron graph. Full declare -> resolve -> cache hit on approved value -> fulfilment dependency -> value handoff -> satisfied promotion. Harness/graff eval passes. Parity + build clean. Measurable debloat.

> ***Value/economics layer (shipped this arc).*** *The substrate's value layer is the* ***reward-not-slash cooperative commons*** *— usage fees -> USDC to used + verified maintainers, forfeit->honest, nothing burned — built + proven on* `arc-lean` *as ~18 zero-dependency modules. Canonical spec:* `ECONOMICS-cooperative-commons.md`*. It* ***coexists with (does not supersede)*** *the canonical Vine Doctrine (slash-to-burn); see DOCTRINE.md ->* `## The Cooperative Commons (arc-lean)`*.*

---

0. Vision & Invariants (from SKILL.md + conversation)

The substrate is a **fractal while-loop over contract neurons**. Every truth claim is a neuron. Pointers (`contract:<id>`) are the universal edge. The cache is pointer-chain keyed (signatures, not raw values). HOLD is non-terminal. The canonical spec is the truth-root.

New mechanics we are adding (all reduce to existing primitives):

- **Pointers surface approved values**: Once a target has terminal `output_signature` and is satisfied (approved), a `contract:<id>` reference (or `value-of:<id>`) can return the concrete result bytes, not just "pass".
- **Self-fulfilling returns**: A contract can return/declare a dependency on another contract and stay in HOLD until that dependency resolves *and* its surfaced value makes the parent claim hold ("equal" or evaluator-defined fulfilment).
- **Every primitive is a contract neuron**: `neuron`, `funnel`, `sequence`, `quorum`, `sabbath-eval`, Genesis sequencing, `flex:*`, `bun:`, `python:`, harnesses, etc. become declared contracts pointing at evaluator shapes. The binary becomes a thin uniform dispatcher + leaf adapters.
- **Strong inheritable Contract base**: A canonical `Contract` / `Neuron` view (typed row + views) so identity, lineage (HKDF signatures), pointer chains, Genesis ordering, cache behaviour, value surfacing, propagation (`enqueueAffected`), W6H, layer/shape, and fulfilment are base properties. New shapes inherit by conforming (composition + comptime assertions). No more giant `if (ptype)` ladders.

These changes enable:

- Cross-claim sharing without violating per-id pointer cache (via explicit fulfilment wiring).
- Proactivity (a waiter contract keeps the drill/resolveLineage alive).
- Massive debloat (collapse `inferPointerType`, `runAction` branches, Genesis tables, ad-hoc prefixes).
- Better brain simulation (uniform neurons firing via evidence + evaluators).

All changes respect:

- Pointers over payload (signature chain remains the cache key; values ride along only after approval).
- Append-only ledger.
- No scripts (new "primitives" are contracts, not new files in scripts/).

- Decalogue parity + build.sh gate.
- Trinitarian decomposition (interpret/verify/adjudicate can use fulfilment).
- HOLD-is-non-terminal + drill.

---

1. Current State Snapshot (what we are changing)

- Cache: strictly per-id via `computeSignatureChain` (parent_sig + dep terminal sigs). `cache_resolved` pins snapshot. `contract-ref` in `runAction` only returns "pass"/"fail" or primitive cached result string. No general value surfacing.
- Fulfilment: implicit via child satisfaction + aggregation. No first-class "I return a contract and wait until its value equals my expectation".
- Primitives: massive dispatch in `pointer.zig:inferPointerType/inferLayer/isKnownAction` (~70-80 lines + duplicates), `zig/src/main.zig:runAction` (~650+ lines of ptype ifs + prefix handlers for neuron/funnel/sequence/quorum/sabbath-eval/drill/agent-judges/bun/python/flex/depends_on/nanobrew/harness/GenesisDay tables/cmdGenesis/currentDay/etc.).
- Abstraction: loose `Row` bag + `Shape` (declared only) + `Synapse`. No named inheritable `Contract`. Two `Row` defs. `pattern.zig` does partial comptime layer enforcement.
- ARC/AGI: none in tree. Branch "arc-lean". Existing bench harness (`docs/bench.sh`) + graff eval + .graff/sessions. No grid tasks.

Debloat opportunities identified (will be executed):

- Collapse `inferPointerType` to small carrier list + base neuron.
- Delete/replace most `runAction` ptype branches with "delegate to declared evaluator contract".
- Remove `GenesisDay/cmdGenesis/currentDay/countDayChildren` (express as normal `contract:<prev>` + day contracts).
- Remove ad-hoc `bun:`, `python:`, `nanobrew:`, `depends_on:`, most `flex:*`, `harness:` special paths (become adapter contracts).
- Shrink `inferLayer`, `actionFromPlan`, various `has*Row`, `bioIsReflex` guards.
- Net: ~800-1200+ lines from Zig binary + helpers; more uniform.

---

2. Phases (execute in order; each has verification gate)

Phase 0 — Plan & Scaffolding (this document)

- Write this MASTER-PLAN.
- Define minimal ARC-AGI-3 validation tasks (see §3).
- Add todo tracking + success criteria.
- Gate: plan reviewed (this file exists, todos populated).

Phase 1 — Strong Contract Base + Value Surfacing on Approved Pointers

Goal: Introduce typed `Contract` / `Neuron` views. Any satisfied terminal contract can surface its concrete result via `contract:<id>` (or `value-of:`) once approved.

Work:

- libcontract: Define `Contract` struct (id + declared view + terminal view carrying `output_signature` + `result` + `satisfied` flag). Helper `surfaceApprovedValue(allocator, env, id) ?[]u8`.
- Unify `Row` (remove duplicate in fractal.zig; make zig/src/main.zig use libcontract view where possible).
- Update `cache.zig` readers + `computeSignatureChain` to work over Contract views (no behaviour change to keying).
- In `runAction` (zig): generalize the `contract-ref` branch:

- If target terminal + has `output_signature` + satisfied -> return the actual `latestActionResult` / surfaced result (not just "pass").
- Emit evidence: "result: PASS — contract:{id} value surfaced ({n} bytes)".
- Add `value-of:<id>` as alias/synapse kind for clarity (maps to same pointer mechanism).
- Update `aggregateChildOutputs` + `break_above` to prefer surfaced values.
- Update `dependentIds` / `enqueueAffected` / `resolveLineage` to treat value-surfacing dependents correctly (they wake on terminal of target).
- Comptime hook: add to `pattern.zig` a check that any new shape declares it participates in value surfacing when terminal.
- Parity: add claim for "approved pointers surface values" (build.sh).

Verification gate:

- Existing tests / bench still pass.
- Manual: declare A (satisfied with result "42"), declare B with action `contract:<A-id>`; B's resolve returns "42" (or the exact result) and satisfies.
- Cache hit path still works (signature chain unchanged).

Debloat note: this phase already lets us delete the "only for primitives" special case in contract-ref.

Phase 2 — Fulfilment / Self-Fulfilling Contract Returns ("until equal")

Goal: A contract can declare/return a dependency contract and only resolve when that dep is terminal *and* its surfaced value satisfies the parent's claim (evaluator-defined "equal").

Work:

- New synapse kind or action form: `fulfil:<id>` or plan that contains "result must equal output of contract:<id>".
- In resolver / `runAction` for neuron/funnel shapes (or a new base "fulfilment-eval" evaluator contract):
- Collect fulfilment deps.
- If any fulfilment dep not terminal -> HOLD (with evidence).
- If terminal, surface its value, run a small equality / predicate evaluator against parent's expectation (or child's output).
- On match -> contribute to parent's satisfaction.
- Make the "return a contract" pattern first-class: when a child's action result is a `contract:<X>` pointer, the parent treats X as a fulfilment dependency by default (or via explicit `fulfil:`).
- Update cache: a fulfilment dep change invalidates the waiter (via existing `cache_invalidated` + dependent walk; may need explicit "value changed" reason).
- Update drill / backfill to keep pressure on fulfilment frontiers.
- Trinitarian tie-in: `verify:<reading>` can be a fulfilment child; `adjudicate` picks the one whose fulfilment succeeded.
- Add to base Contract: `fulfilmentDeps() []const u8`, `checkFulfilmentAgainst(allocator, value, expectation) bool`.

Verification gate:

- Declare producer P that outputs a grid or scalar.
- Declare consumer C: "must equal P's output" (via `contract:<P>` or `fulfil:<P>`).
- C stays HOLD until P satisfied; then C resolves if values match, else dies or HOLDs for more waves.
- Cache: after first success, re-resolve of C hits if P unchanged.
- Evidence lines show fulfilment checks.

Phase 3 — Primitives Become Contracts (Debloat Execution)

Goal: Collapse the special-case explosion. `neuron`/`funnel`/`sequence`/`quorum`/Genesis/etc. are declared contracts with evaluator shapes.

Work (biggest source of lines removed):

- `pointer.zig`:
- Shrink `inferPointerType` to: core carriers (contract:, posthook:, wrapper:, sha256-*, tunnel:, webhook:, at:, repeat:, word:, validate:, signed:) + "neuron" (default for graph shapes) + small legacy aliases.
- Update `isKnownAction`, `inferLayer` in lockstep.
- Remove or deprecate many old prefixes (they parse for backward compat but no longer have special branches).
- `zig/src/main.zig:runAction`:
- Keep only: drill (or make it a standing contract), leaf adapters (shell/http as final fallback), sabbath-eval (temporarily or as canonical evaluator contract), agent-judges sentinel (HOLD).
- All other: "lookup declared evaluator contract for this shape (via action or a `shape:<name>` synapse), resolve its children, surface aggregated + value results".
- Delete: the 100+ lines of funnel/sequence/quorum/funnel-first/child checks, bun:/python:/nanobrew:/depends_on:/harness:/flex: special handlers (replace with declared contracts that call the same leaf exec).
- Remove / deprecate:
- `GenesisDay`, `days_table`, `cmdGenesis`, `currentDay`, `countDayChildren`, `findSessionByPlan` (Genesis sequencing expressed via `contract:<dayN-1>` + day contracts).
- Ad-hoc `hasSabbathEvaluationChild` / many `has*` guards (move logic into evaluator contracts or base Contract methods).
- Large parts of `actionFromPlan`, `bioIsReflex`.
- Bootstrap canonical evaluator contracts on first run or via build (e.g., declare "evaluator:neuron", "evaluator:fulfilment", "evaluator:funnel" etc. with plans describing their rules; the binary knows the minimal set to seed).
- Update `shape.zig` validation to accept the reduced known-action set + any declared shape.
- Update `pattern.zig` SAMPLE_PREFIXES and assertions (now most "primitives" are just neurons with specific synapses).
- `aggregateChildOutputs`, `break_above`, value surfacing remain (they are base Contract behaviour).

Debloat target: 800-1200+ lines removed from Zig sources + tests. `runAction` drops to <200 lines. `inferPointerType` to ~20-25 lines.

Verification gate:

- Rebuild + parity passes (new claim: "primitives are contracts").
- Existing behaviour for neurons/funnels/sequences preserved via the new evaluator contracts.
- Old special actions (if any rows) still parse (compat) but new code uses contract form.
- Bench numbers do not regress (or improve due to less branching).

Phase 4 — Full Inheritance: Base Contract Properties for Everything

Goal: `Contract` / `Neuron` is a strong, inheritable abstraction. New things (adapters, custom evaluators, the drill itself, value carriers) get identity/lineage/pointer-chain/cache/fulfilment/propagation/value-surfacing "for free".

Work:

- libcontract: Promote `Contract` (from Phase 1) to first-class exported type with methods:
- `id()`, `parentId()`, `synapses()`, `layer()`, `shape()`.

- `signatureChain()`, `isTerminal()`, `surfaceApprovedValue()`, `fulfilmentDeps()`, `checkFulfilment()`.
- `dependsOn()` / `dependents()` (via ledger walks).
- `latestResult()` (when approved).
- zig main + libcontract: Make `resolveOne`, `runAction` (now thin), `enqueueAffected`, cache check, value surfacing, etc. operate on `Contract` views.
- Comptime: enhance `pattern.zig` (or new `contract_trait.zig`) with required properties:
- comptime assert every declared shape participates in signature chain, can surface value when terminal, answers W6H, etc.
- Make "evaluator contracts" and "adapter contracts" first-class `Neuron` subtypes (via composition: they embed base Contract + add `evaluatorPlan` or `adapterKind`).
- Update ledger readers (`parseRows`, `latestDeclared`, `isSatisfied`) to return/produce `Contract` where possible.
- Update SKILL.md sections: Neuron Shape, Pointer-keyed cache, Continuous resolver, Public Shape, to name the inheritable base.
- Add parity claims for the abstraction (every new contract mechanism must be expressible as a Contract with the base properties).
- Docs: add small diagram "Contract base -> Neuron / Evaluator / Adapter / Fulfilment all inherit".

Verification gate:

- A custom adapter or evaluator declared as a plain contract automatically gets cache, value surfacing, propagation, etc. without extra code.
- No behaviour change for legacy paths.
- `aiko` binary size reduced; complexity of adding a new "primitive" is now "declare a contract + evaluator row".

Phase 5 — ARC-AGI-3 End-to-End Validation + Harness Integration

Goal: Express 2-3 small ARC-style tasks as contract declarations. Use the full new machinery (neurons, value-surfacing pointers, fulfilment deps, cache hits on approved solutions). Run complete substrate loop to satisfied + correct output. Score via existing graff/harness.

ARC tasks (minimal, self-contained, reproducible):

- Task A (core): "Copy color of single object to all cells of same shape" on 3x3 or 4x4 grids. Input grid + expected output grid declared. A "rule neuron" child must produce the output grid. Fulfilment: consumer checks equality with expected.
- Task B: "Count distinct colors" or simple symmetry completion. Use funnel/sequence of sub-neurons.
- Task C (stretch): A small "program synthesis" where one contract emits a tiny rule description and another fulfils by executing it on a held-out grid.

Implementation:

- Add `examples/arc-agi3/` (or under references/) with tiny JSON grids + plans.
- Declare via `aiko` (or test harness) the root claim + child neurons + fulfilment links.
- Use `contract:` / `value-of:` / `fulfil:` synapses.
- Expect: declare succeeds, resolver runs (possibly multiple waves), cache hit on second identical solve, fulfilment passes, root satisfied with correct grid as result.
- Evidence lines show value surfacing + fulfilment checks.
- Integrate with `docs/bench.sh` style or new `arc-validate.sh` that:
- Seeds a fresh ledger.
- Runs the task declares (private).
- Runs backfill/drill until settled.

- Checks terminal satisfied row + exact result matches expected.
- Records metrics (waves, cache hits, bytes surfaced).
- Run `graff --eval` (or equivalent harness command) on the session/output. Capture score.
- Update `.graff/sessions/` or create a dedicated ARC session log.

Success criteria (all must pass):

- At least one task fully solves end-to-end using new fulfilment + value surfacing.
- Cache hit observed on re-resolve (pointer chain stable).
- No special-case branches for the ARC neurons (they are plain "neuron" or declared evaluator contracts).
- Build + parity clean.
- Harness/eval records a passing result (or explicit "ARC-AGI-3 minimal solved" marker).
- Debloat measurable (run `cloc` or `wc -l` before/after on zig/src + libcontract/src; note reduction in pointer.zig + main.zig dispatch).

---

3. ARC-AGI-3 Minimal Task Definitions (for execution)

(Will be materialized as files + aiko declares during Phase 5.)

Example root plan:

```
arc-task: copy-color-on-shape — given input grid with one colored object, output grid with that color copied to all cells of the object. Use contract neurons. Fulfil against expected.
```

Children:

- interpret: grid parse
- neuron: detect object (contract: rule detector)
- neuron: apply color (contract: applicator)
- verify: output equals expected (fulfilment dep on applicator result)
- adjudicate: promote the solved grid

The substrate must surface the final grid as the satisfied result of the root.

---

4. Cross-Cutting Requirements

- **SKILL.md updates**: Add or extend sections for "Value-surfacing pointers", "Fulfilment dependencies", "Primitives as contracts", "Contract base inheritance". Cite the relevant doctrine/spec where appropriate. Add to the eight-instance table if "foundation precedes creativity" applies.
- **build.sh parity**: New numbered claims for the four mechanics. Decalogue gate still runs. No new scripts/ files.
- **No env var regressions**: All new behaviour via goals/contracts only.
- **Backward compat**: Old rows with special actions still parse and run (at least until a migration contract is declared).
- **Tests**: Add unit tests in libcontract for new Contract view + value surfacing + fulfilment predicate. Integration via bench or new arc-validate.
- **Evidence / observability**: Every value surface and fulfilment check emits canonical evidence line. Silent fallbacks forbidden.
- **Performance**: Cache hit rate must not regress; fulfilment should not cause unnecessary re-execution (use signature chain + value snapshot).

---

5. Execution Order & Gates

- Write this plan (done when committed/visible).

- Phase 1 implementation + gate.
- Phase 2 + gate.
- Phase 3 (debloat) + gate (rebuild, parity, size diff).
- Phase 4 (inheritance) + gate.
- Phase 5 (ARC tasks + full E2E + harness eval) + final gate.
- Update docs, SKILL.md, CLAUDE.md pointers.
- Final build + sign (via scripts/build.sh) + graff eval record.
- Mark plan complete with summary in this file (or a RESULTS section).

Rollback: each phase must be bisect-friendly; keep compat shims until ARC validation succeeds.

---

6. Metrics of Success

- Functional: ARC minimal task(s) solved correctly via the substrate using fulfilment + surfaced values + cache.
- Structural: `inferPointerType` and `runAction` dispatch size reduced by >60%. No new special branches for shapes.
- Inheritance: Adding a new evaluator or adapter requires only declaring contracts + (optionally) a leaf adapter binding. It automatically gets cache, lineage, value surfacing, propagation.
- Quality: Parity passes, no new scripts/, evidence lines for all new paths, HOLD non-terminal preserved.
- Harness: `graff --eval` (or bench/arc-validate) reports clean pass for the ARC session.

---

7. Open Risks & Mitigations

- Row growth from making everything a contract -> use existing inputs_hash dedup + cache + hebbian primitive promotion.
- Migration of old special rows -> keep parsing compat + declare a migration contract if needed.
- Performance of fulfilment walks -> rely on existing dependentIds + cache invalidation; profile if needed.
- ARC task complexity -> start with 1-2 trivial grids (3x3); scale only after core loop works.

This document is the single source of truth for the work. When executing, update the status of each phase here (or append a RESULTS section at the end). All changes must be justified against the invariants above.

RESULTS (live)

Phase 1 (value surfacing on approved `contract:<id>`*)*

- Generalized `runAction` contract-ref branch: any terminal with `latestActionResult` now surfaces bytes + "value surfaced (N bytes)" evidence.
- `arc-validate.sh` (Phase-1) exits 0 (value surfaced observed or bytes present).
- Build + parity clean. Signed binary updated.

Phase 2 (fulfilment / self-fulfilling returns)

- Added `fulfil:<id>` and `value-of:<id>` pointer kinds (pointer.zig + isKnownAction + inferLayer + dependentIds).
- `runAction` branches: `fulfil:` waits (HOLD) until target terminal then surfaces its bytes; `value-of:` is explicit alias.
- `dependentIds` walks `fulfil:` and `value-of:` in addition to `contract:`.
- `bioIsReflex` recognizes them for local short-circuit.

- New `examples/arc-agi3/arc-validate-fulfil.sh` exercises producer->fulfil-consumer; exits 0.
- Evidence lines: "fulfil:... value surfaced", HOLD while pending, PASS on surfacing.
- Build + parity clean after Phase-2 edits.

Harness correction (2026-07-14) — why Phases 1-2 looked broken

Both validators were wrong in ways that made the substrate look broken while it was not. The Zig was correct; the tests were lying in both directions.

- **Ledger isolation was by `$HOME`, but `ledger.zig:resolvePath` resolves by `cwd`.** It walks `<cwd>/.claude/contracts.jsonl` first and only falls back to `$HOME/.contracts/` — never `$HOME/.claude/`. The scripts exported `HOME=$tmp` and ran from the repo root, which has a `.claude/`. So every "isolated" run actually read and wrote the repo's own 37MB / 50k-row ledger, while the rows the test appended went to a `$tmp` file nothing ever read. That is the whole explanation for the invisible satisfied producers, the random HOLDs, and the run-to-run flakiness. Fix: `cd` into the sandbox.
- **The contract id was parsed from merged stdout+stderr with `tail -c 8`.** The id is a bare 8-hex line on *stdout*; evidence goes to *stderr*. `tail -c 8` returned the last 8 characters of whatever prose printed last — e.g. `...frontier stuck` yielded the id `ierstuck`. Every `fulfil:` under test pointed at an id no contract ever had. Fix: read stdout, match `^[0-9a-f]{8}$`, and fail loudly when absent instead of inventing a random id.
- **The assertions could not fail.** `arc-validate.sh` passed if `ARC_OK_42` appeared *anywhere* in the ledger — which it always did, because the script appended it to the producer itself. Phase 1 was reported green while asserting nothing. Fix: assert only on the **consumer's own rows**.

Rewritten: `examples/arc-agi3/_harness.sh` (isolation + id parsing + honest assertions), consumed by both validators.

Phases 1-2: VERIFIED (not just "mechanism exercised")

With an honest harness, the consumer's own ledger rows show the full handoff:

```
declared        action='fulfil:3664117f'   result=None
iterated        action=''                  result='ARC_OK_42'   <- producer's bytes adopted
satisfied                                                        <- consumer resolved true
cache_resolved                                                   <- pointer-chain snapshot
```

- Phase 1 (`contract:<id>` surfacing) — PASS, 3/3 repeatable.
- Phase 2 (`fulfil:<id>`) — PASS, 3/3 repeatable, and now also asserts the HOLD half: a waiter on an *unsettled* producer must block, not pass.
- Mutation-tested: removing the producer's satisfaction, or corrupting the surfaced bytes, turns both validators red. They can fail.

Phase 3 (debloat): premise was false — 149 lines, not 800-1200

The plan estimated 800-1200 removable lines. Measured against reality, that estimate does not survive.

Actually deleted (provably dead, 149 lines): the Genesis island — `GenesisDay`, `days_table`, `findSessionByPlan`, `currentDay`, `countDayChildren`, `DayCount`, `cmdGenesis`. `cmdGenesis` had **zero call sites**, and the `--genesis` verb that once invoked it is *forbidden by parity claim "no verbs"* (build.sh:322 fails the build if the literal appears). Dead code kept alive by nothing. Removed; 13/13 tests, 50/50 parity, verify-self and verify-binary all still green, both validators still PASS.

Everything else on the hit-list is NOT dead. Two measurements say so:

- *Ledger usage.* Across 50,276 rows of real history, the only action kinds that have EVER occurred are `neuron` (756), `sabbath-eval:` (62), `sabbath:` (12), and `fulfil:` (3). Zero rows for `bun:`, `python:`, `flex:*`, `nanobrew:`, `harness:`, `depends_on:`, `quorum:`, `funnel`, `sequence`, `cli`, `api`, `contract:`.
- *Doctrine pinning.* **[CORRECTED 2026-07-14 — the first version of this section was WRONG.]** I originally claimed every zero-usage kind was declared doctrine, so none could be deleted. That was an artifact of a naive substring test: matching `"agent"` against SKILL.md hits prose like *"the agent judges the report"*, not

the `agent:` pointer. An adversarial audit refuted it, and a literal-token probe (with controls: `quorum` 12 hits, `wrapper:` 2, `blocked_by` 8) confirmed the refutation.

Seven pointer kinds are zero-usage in the `action` field AND undeclared in doctrine. **[CORRECTED AGAIN — they are still NOT deletable. None of them.]**

"Undeclared + unused" turned out to mean nothing. Each kind is load-bearing for a reason no ledger census or doctrine grep can see:

kind	why it must stay
`sha256-private` / `sha256-shared` / `sha256:`	`main.zig:3858` **PRODUCES these prefixes** for stderr-tape blob evidence. 384 + 23 rows carry them — in the `pointer` field, not `action`, which is why every census missed them. Delete -> the resolver classifies a content hash as `cli` and **shell-execs it**; `shape.zig:validateShape` rejects the envelope.
`agent-judges`	Shell-exec guard + two `backfillUnresolved` skip-filters. `main.zig:1847` records what happened last time it fell through: `sh: agent-judges: command not found` on every backfill pass. Its zero rows are the *evidence the guard works*.
`agent:`	**Sole layer-6 inhabitant.** `pattern.zig:assertInvariants()` (eagerly invoked from `contract.zig:65`) `@compileErrors` if any creation-day layer 1..6 empties. Deleting it does not fail a test — it fails the *compile*.
`machine:`	**Sole layer-1 inhabitant.** Same comptime invariant.
`funnel-first`	Live `bioIsReflex` + `runAction` branch — a working feature that is merely unexercised.
`embedding:`	The only genuinely inert kind. Not worth a separate cut.

Three independent audits said "safe to delete." All three were wrong, each in a different way: one matched substrings, one counted only the `action` field, one replayed the dispatch chain but never asked who *writes* the prefixes. The kinds that look deadest are the guards, and a guard's whole job is to leave no trace.

Phase 3's real yield is 149 lines, and it is finished. The Genesis island was dead (zero callers, its invoking verb already outlawed). Nothing else in the dispatch surface is. The MASTER PLAN's 800-1200 line estimate was fiction.

Genuinely dead, and the one remaining cut available: `libcontract/src/action.zig` — a complete shadow implementation of the dispatch table, imported by nothing and absent from `build.zig`'s `test_files`, so it is never compiled. It is also uncommitted/untracked. Deleting it is safe; it is not part of the substrate.

Signal installed (this is the durable outcome): a regression lock in `zig/src/pointer.zig` asserting the generated + sentinel kinds never degrade to `cli`. It passes on correct code and goes red the moment the guard is removed — verified by mutation. The next agent that comes here to "shrink the dispatch table" will be stopped by a failing test instead of by a corrupted ledger.

Special case — `value-of:` **is NOT dead.** It is undeclared only because *we invented it this session* (Phase 2). The correct action is to **write its doctrine**, not delete it.

Still genuinely promised (declared in SKILL.md, so NOT deletable without a doctrine amendment): `quorum`, `funnel`, `sequence` (SKILL.md:998 — "cells, funnels, loops, harnesses, quorums, sequences"), `harness`, `loop`, `contract-ref`, `inhibits`, `wrapper:`, `word:`, `validate:`/`signed:`, `wallet:`, `iterate:`, and the four negatively-pinned synapse kinds (`blocked_by`, `prehook`, `bind`, `modality` — build.sh:941-954 pins them as kinds cache.zig must NOT walk).

Also corrected: `cli` is **live** (832 ledger rows), not zero — the first measurement's prefix-splitter mis-bucketed it.

Lesson: the substring heuristic that produced the false claim is the same class of error as the harness bugs it was diagnosing. Assertions that cannot distinguish a real hit from a coincidence are worthless whichever direction they point.

Adversarial pass (Step 5-6) — one real bug found and fixed, one guarantee confirmed

Ran the mechanism under real conditions (dead/purged targets, self-cycles, two-hop chains, a hostile-payload shell-exec attack, cache re-resolution).

- **CONFIRMED CONTAINED — surfaced values never execute.** An 81-byte hostile payload (`$(touch …); …`) surfaced through `contract:`/`fulfil:`/`value-of:` was stored verbatim as `action_result` DATA; no marker file, no shell. There is a hard split: `action` is executable, `action_result` is data, and nothing

re-promotes one to the other. (The Step-4 guard comment claimed the sha256 branch "prevents shell-exec" — that was wrong and is now corrected in pointer.zig: a sha256 *action* execs like any cli action; the branch is for classification, not execution. The real guarantee is the action/result split.)

- **FIXED — the purged temporal-incoherence bug.** `isSatisfied` (main.zig:509) walked only `satisfied`/`died` and was blind to `purged`. `cache.zig:388 terminalClass` already treats purged as `.null_terminal`, and `contract.zig`'s markPurged docstring says the terminal walk must — but `isSatisfied` (the read surface the fulfil/contract-ref/value-of branches call) was never updated. So a **satisfied-then-purged** producer surfaced its stale bytes and resolved a consumer to PASS. Same class as the `died` fix (contract a1c68c4e). One line added; regression test `examples/arc-agi3/arc-validate-purge.sh` fails without it (mutation-verified) and passes with it.
- Follow-up noted, not done: `isDead` (main.zig:517) is also blind to purged; it feeds the kill-lineage guard. Left alone — unverified, out of scope this pass.
- **No hazard** on cycles (bounded settle loops -> clean HOLD, no hang) or multi-hop value propagation (each hop re-persists bytes via recordIteration).
- **action.zig deleted** — the shadow dispatch table, untracked, imported by nothing, absent from build.zig. Behaviour-neutral (build + all validators green). The genuinely-dead code the debloat was actually looking for.

Current status

- Behavioral core live **and verified**: `contract:` / `fulfil:` / `value-of:` surface approved terminal values; fulfilment deps HOLD until the target is terminal, then adopt its bytes and resolve.
- Cache remains pointer/signature keyed (no semantic `1+1 == 2+0`).
- Debloat (Phase 3), strong Contract base (Phase 4), full ARC grid E2E (Phase 5), and SKILL.md updates remain — none of them started.
- Note: ~70 rows of test spill leaked into the repo ledger before the isolation bug was found. Harmless; prune if desired.

Phase 5 (ARC-AGI-3 end-to-end): DONE — the substrate solves grid tasks

The Success Target is met. `examples/arc-agi3/arc-task.sh` drives grid tasks **through the substrate's own resolution**, and the test asserts only that the surfaced result equals the task's expected output — it never computes or injects the answer:

```
declared   action='python:g=[[1,2,3]...];print(transpose(g))'   <- solver neuron
satisfied  action_result='[[1,4,7],[2,5,8],[3,6,9]]'           <- leaf computed it
declared   action='fulfil:<solver>'                            <- root adopts
satisfied  action_result='[[1,4,7],[2,5,8],[3,6,9]]'           <- correct grid surfaced
```

- Full loop: declare -> resolve (a `python:` leaf computes inside resolution) -> fulfilment value-handoff -> root satisfied with the correct grid -> stable on re-solve.
- Two structurally-different tasks solved: `transpose-3x3`, `reflect-h-3x3` (self-describing fixtures in `examples/arc-agi3/tasks/*.json`: input, expected, and a quote-free python `solver`).
- **Mutation-proven**: a wrong solver output turns the runner red (verified).
- Runs **offline** via the `contract` persona — `aiko` is the remote LLM-compile persona, so leaf execution + backfill live in the native binary. This is a real finding the plan did not anticipate: the local declare path cannot mint a leaf-executor action (`actionFromPlan` recognizes only pointer/graph prefixes; leaf actions arrive via hand-declared rows that backfill runs without re-validating, or via the cloud compiler).
- Commit `b9da6671`. 9/9 arc validators green (8 mechanics + this E2E).

Not attempted (honestly out of scope): autonomous *rule inference* (the solver rule is given, not learned) and multi-hop compute-chaining where one neuron's output is fed as another's *input* — the substrate adopts a value as a consumer's *result*, it does not inject it as a downstream *input*. The E2E proves orchestration (declare->compute->verify->surface), not program synthesis.

Phase 4 (strong inheritable Contract base): RETIRED — not built, by design

Retired under the question->delete lens, for the same reason Phase 3's estimate collapsed. Phase 4's core promise — "inheritance eliminates the if(ptype) dispatch" — inherits Phase 3's **disproven** premise: the dispatch is load-bearing (guards, sole-layer inhabitants, comptime invariants), not bloat to abstract away. The one concrete residue, "two `Row` defs," is not a defect: `main.zig`'s Row is the 20-field resolution struct; `fractal.zig`'s is a 6-field **display** struct. Unifying them would couple the display module to the full resolution struct — a regression, not a simplification. The resolvers (`resolveOne`, `runAction`, cache, value surfacing) are working and verified (terminal axis, receptor, F8, ARC E2E all green); rewriting them onto a Contract abstraction is gold-plating a proven core. Not built.

Phase 3 doctrine follow-up: DONE — `value-of:` documented

`value-of:` (invented in Phase 2) is now written into SKILL.md (the "opt-in receptor" section documents `contract:`/`fulfil:`/`value-of:` value adoption), so the RESULTS' "undeclared" flag is cleared.

Substrate hardening (fork census, same branch)

The substrate these phases run on was reconciled to one-authority consistency this session (committed on `arc-lean`): terminal/death recognition (`terminal.zig`), the F1 opt-in receptor (`capability.zig`), the F8 shell-exec collapse (one classifier + default-deny — which correctly HELD the `word:` fanout during the ARC drills instead of shell-executing it), and F9/F10 hygiene. Build gate green at 53 parity claims, all validators mutation-proven.

FINAL STATUS — plan complete at its true scope

Phase	Status
1 value surfacing	DONE + verified (hardened: terminal authority + F5 death-gate)
2 fulfilment	DONE + verified
3 debloat	DONE at true size (149 dead lines); 800–1200 estimate was fiction
4 strong Contract base	RETIRED — premise disproven; would regress, not improve
5 ARC-AGI-3 E2E	DONE — substrate solves grid tasks end-to-end, mutation-proven

The through-line of the whole plan, learned the hard way: **measure before you build.** Every over-estimate here (the 1200-line debloat, the mandatory-receptor intuition, the strong-base abstraction) fell to a measurement. What survived was small, load-bearing, and verified.

End of plan.